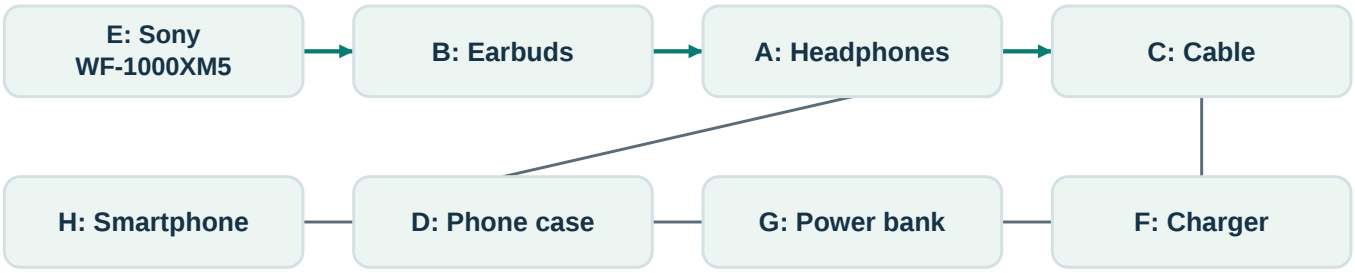
Combining Walk-Text and Label Deconvolution Embeddings for SAGN-Based Product Node Classification

Junhao Shen, Rishi Puri, Meghana Shanthappa

The task is node classification: predict a category for each product.

One product graph throughout



Undirected toy links; teal arrows mark the selected walk E - B - A - C.

All names, descriptions, links and small numerical examples in this report are **illustrative**. The Sony product name is real, but these are not verified OGB items or observed co-purchase links. The actual benchmark has 47 categories; two-category examples only make the arithmetic easier.

Input for each product	Size	What it supplies
Released LD-BERT embedding	768 values	Node text trained with graph-aware label supervision
Frozen walk-text embedding	768 values	A frozen LM jointly reads root and visited-product text
Raw OGB feature vector	100 values	The benchmark's original node attributes
Concatenated input	1,636 values	Keep all three feature groups; do not average them

Why combine LD and walk-text?

LD supplies task-oriented text features learned with graph-aware supervision. Walk-text supplies context from linked-product descriptions. Keeping both lets SAGN learn from their combined information. Their different views **may be complementary**; a separate ablation is needed to measure that benefit. [2, 3, 8]

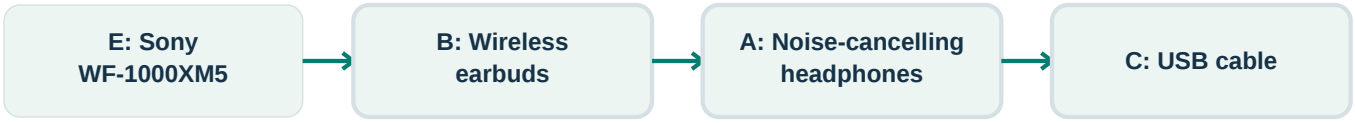
Product text + graph -> LD / walk-text feature vectors -> **SAGN/SLE** -> class predictions -> **Correct & Smooth** -> refined class predictions. [8]

Reading guide

Walk-text: page 2. LD: pages 3-4. Feature combination and SAGN/SLE: page 5. Correct & Smooth: pages 6-7. The actual combination: page 8. Terms and sources: page 9.

1. Walk-text: paths become vectors

Read the product's text together with text from products reached along graph links.



Step 1 - choose a walk

Start at Sony earbuds E. One possible walk is **E - B - A - C**: Sony earbuds, generic earbuds, headphones, then a cable. This is **four visited nodes and three edge transitions**. Another possible route is E - B - A - D. A random walk can revisit a node.

Step 2 - render the visited attributes as text

Schematic input: "Sony WF-1000XM5 earbuds" | "wireless earbuds" | "noise-cancelling headphones" | "USB cable"

The product name is an attribute; E is just its node identifier. No category labels are inserted into this walk text. The schematic above is for readability: the reconstruction code actually separates node text with the literal word **cites**, even for products. [8]

Step 3 - encode, pool and normalize

Your pipeline uses a frozen **google/embeddinggemma-300m** model. It averages final token states using the attention mask, then L2-normalizes the result. Each product receives one 768-value vector. This context vector stays separate from the LD vector until concatenation. [8]

Tiny pooling example	Value
Three real token states; padding ignored	[1, 0], [0, 2], [2, 1]
Masked mean	[(1+0+2)/3, (0+2+1)/3] = [1, 1]
L2 normalization	[1, 1] / sqrt(2) = [0.7071, 0.7071]

These two coordinates illustrate pooling only. Real token states and embeddings have 768 coordinates, with no fixed coordinate named Audio or Accessories.

Important details of this experiment

Four walks are generated per node, but the **160-token limit retains only the first rendered walk's prefix**; walks 2-4 do not reach the encoder. Up to 160 text characters per visited node are rendered. Thus more generated walks did not mean more encoded context in this artifact. [8]

Original WalkLM fine-tunes a language model on walk-derived text. This project borrows the walk-to-text idea but keeps the encoder frozen: **WalkLM-style, not a reproduction of original WalkLM training**. Freezing reuses existing language knowledge without encoder updates; it does not prove fine-tuning would be worse. [3, 8]

2. LD: train features for graph mixing

LD means Label Deconvolution, not simply Language-Derived.

A graph model later mixes each product's features with neighbor features. LD makes text-encoder supervision account for that future mixing, instead of treating every product as isolated. It learns graph-aware training targets while the stored correct class remains unchanged. Unlike walk-text, LD brings upstream task supervision into the text features; it is not the same encoding step. [2]

A practical-style numerical example

Use two toy classes, ordered **[Audio, Other]**. Suppose these four nodes have known training labels. A has neighbors B, C and D in our graph.

Product	Known class	One-hot label
A: Headphones	Audio	[1, 0]
B: Earbuds	Audio	[1, 0]
C: Cable	Other	[0, 1]
D: Phone case	Other	[0, 1]

A simple neighbor-label mean is ([1,0] + [0,1] + [0,1]) / 3 = **[1/3, 2/3]**. For illustration, choose 70% own-label information and 30% one-hop label information.

Adjusted training target for A
0.7 x [1, 0] + 0.3 x [1/3, 2/3] = **[0.8, 0.2]**

Read the two entries as supervision scores for Audio and Other. They are **not** a new stored label, an observed prediction, or the 768-dimensional embedding. A's correct class is still Audio; the final prediction is evaluated against that original class.

What is learned, and what was reused?

LD training in the method	Your experiment
Combine multi-hop label signals with learned weights; train the text encoder using adjusted supervision.	Reuse the released 768-dimensional LD-BERT features. This report does not claim you retrained LD.
Extract node features, then train a downstream graph model with the encoder fixed.	Concatenate LD features with walk-text and raw features, then train SAGN/SLE.

The 70/30 weights and simple mean are teaching choices, not recovered LD settings. Practical LD uses learned multi-hop label combinations and normalizes targets for classification. Its exact graph normalization need not equal this neighbor mean. [2]

3. Why show [1.5, -0.5]?

A theoretical inverse-mixing illustration, not the practical LD targets or actual released embeddings.

Keep only two connected nodes: **A headphones** and **D phone case**; ignore their other links for this example. Order scores as **[Audio, Accessories]**. Let a toy graph operator mix 75% own scores and 25% neighbor scores.

Scores before graph mixing	Headphones A	Phone case D
Original one-hot values	[1, 0]	[0, 1]
Inverse-adjusted toy scores	[1.5, -0.5]	[-0.5, 1.5]

Using the original values

A: 0.75 x [1, 0] + 0.25 x [0, 1] = **[0.75, 0.25]** -> Audio
D: 0.75 x [0, 1] + 0.25 x [1, 0] = **[0.25, 0.75]** -> Accessories

Using the inverse-adjusted values

A's Audio score: 0.75 x 1.5 + 0.25 x (-0.5) = **1**
A's Accessories score: 0.75 x (-0.5) + 0.25 x 1.5 = **0**
A becomes **[1, 0]**; the symmetric calculation gives D = **[0, 1]**.

The larger positive score and the negative score compensate for later neighbor mixing. This makes LD's inverse-mapping motivation tangible. Both original and adjusted cases already predict A as Audio; the example demonstrates **exact recovery of toy target scores**, not a measured accuracy gain. [2]

4. SAGN/SLE: use the right distance

SAGN = Scalable and Adaptive Graph Neural Networks.

Toy fusion for E: LD [2.0, 0.5] || walk-text [0.6, 0.8] || raw [0.3]
= **[2.0, 0.5, 0.6, 0.8, 0.3]**. Real input: **768 + 768 + 100 = 1,636**.

Toy latent coordinates, not class probabilities; feature blocks are joined, not averaged or semantically aligned.

SAGN precomputes 0-, 1- and 2-hop summaries from the **combined feature vectors**. Each has its own small MLP, followed by **node-specific attention across hops**. A residual and classifier produce class scores. For E, B is 1 hop away and A is reached in 2 hops. [4, 8]

Same eight products: illustrative attention weights

Product	0-hop	1-hop	2-hop
A: Headphones	80%	15%	5%
B: Earbuds	70%	25%	5%
C: Cable	40%	40%	20%
D: Phone case	60%	30%	10%
E: Sony WF-1000XM5	20%	60%	20%
F: Charger	65%	25%	10%
G: Power bank	55%	35%	10%
H: Smartphone	75%	20%	5%

Every row sums to 100%. All values are invented for explanation, not measured attention. These weights mix **encoded hop summaries**, not individual neighbors or class probabilities. Summaries can include own-node information and revisits, not only nodes exactly that distance away.

A numerical mixture for Sony earbuds E

Suppose its three encoded summaries are h0 = [1,0], h1 = [0.6,0.4], h2 = [0.8,0.2]. These are two toy feature coordinates, not Audio/Other probabilities.

0.2 x h0 + 0.6 x h1 + 0.2 x h2 = **[0.72, 0.28]**
The residual and classifier come next; [0.72, 0.28] is not yet the final class prediction.

What SLE adds

Self-Label-Enhanced training combines label propagation with staged self-training. Confident predictions become pseudo-labels for later stages; they are model guesses, not revealed test answers. Your run used four stages. SLE is a training framework around the predictor, not another name for SAGN. [4, 8]

5. Correct: spread known errors

Correct & Smooth refines class predictions after the model. It does not retrain either text encoder. [5]

Teaching subgraph: A-B, A-C, A-D, B-E only. Ignore F/G/H and their links. Use neighbor means without self-loops, alpha = 0.5, scale = 1, and two iterations per stage. These are not the actual experiment settings.

Start with predictions and only two known labels

Product	Known training label	Base [Audio, Other]
A: Headphones	Audio: [1,0]	[0.70, 0.30]
B: Earbuds	Unknown	[0.60, 0.40]
C: Cable	Unknown	[0.20, 0.80]
D: Phone case	Other: [0,1]	[0.10, 0.90]
E: Sony WF-1000XM5	Unknown	[0.40, 0.60]

Only A and D provide true labels. Their residuals are **label - prediction**:
A: [1,0] - [0.70,0.30] = **[+0.30,-0.30]**
D: [0,1] - [0.10,0.90] = **[-0.10,+0.10]**.

Initialize B/C/E residuals at zero because their labels are unknown, not because their predictions are perfect. Propagate these errors, then add the result to each original prediction. [5, 6]

At each pass: **new = 0.5 x mean(neighbors' current residuals) + 0.5 x fixed residual seed**. After each correction pass, restore A and D to their known residual seeds. [6, 7]

Audio residual values

Node	Seed	Pass 1	Pass 2	Corrected Audio
A: Headphones	0.3000	0.3000	0.3000	1.0000
B: Earbuds	0.0000	0.0750	0.0750	0.6750
C: Cable	0.0000	0.1500	0.1500	0.3500
D: Phone case	-0.1000	-0.1000	-0.1000	0.0000
E: Sony WF-1000XM5	0.0000	0.0000	0.0375	0.4375

For B: 0.5 x mean(A: 0.30, E: 0) = **0.075**. On the next pass, E receives 0.5 x 0.075 = **0.0375**. Its corrected Audio score becomes 0.40 + 0.0375 = **0.4375**. The Other residual is the negative of the Audio residual in this two-class example.

6. Smooth: spread prediction scores

Correct spreads residuals; Smooth spreads label/prediction scores.

Start from the corrected predictions. Replace A and D with their known one-hot training labels once before smoothing. In this fixed-scale teaching variant, they are **not clamped after each smoothing pass**. [6]

new = 0.5 x mean(neighbors' current scores) + 0.5 x fixed smoothing seed
The restart term always uses the initial smoothing seed, not the previous iteration. [7]

Follow Sony earbuds E

E has only neighbor B in this five-node subgraph. E's fixed seed is 0.4375 and B's seed is 0.675.
Pass 1: 0.5 x 0.675 + 0.5 x 0.4375 = **0.55625**.
After pass 1, B is 0.696875.
Pass 2: 0.5 x 0.696875 + 0.5 x 0.4375 = **0.5671875**.

Node	Fixed seed	Smooth pass 1	Smooth pass 2
A: Headphones	1.000000	0.670833	0.811979
B: Earbuds	0.675000	0.696875	0.644271
C: Cable	0.350000	0.675000	0.510417
D: Phone case	0.000000	0.500000	0.335417
E: Sony WF-1000XM5	0.437500	0.556250	0.567188

Before and after the complete toy procedure

Product	Base	After Correct	After Smooth
A: Headphones	[0.700, 0.300]	[1.000, 0.000]	[0.812, 0.188]
B: Earbuds	[0.600, 0.400]	[0.675, 0.325]	[0.644, 0.356]
C: Cable	[0.200, 0.800]	[0.350, 0.650]	[0.510, 0.490]
D: Phone case	[0.100, 0.900]	[0.000, 1.000]	[0.335, 0.665]
E: Sony WF-1000XM5	[0.400, 0.600]	[0.438, 0.562]	[0.567, 0.433]

E switches from Other to Audio, but its true class was never supplied. That change alone is **not evidence of better accuracy**. Cable C also moves slightly toward Audio; that could be wrong if its true class were Other. Cross-category links can make smoothing harmful.

Keep the stages separate. LD changes encoder supervision during upstream training. SAGN learns feature/hop combinations. C&S changes prediction scores afterward. Only training labels may seed this example.

7. The LD + walk-text combination in the actual experiment

The examples above are simplified; the complete real pipeline is summarized here. [8]

Separate views, one downstream predictor

The released LD branch and frozen walk-text branch are prepared separately, then joined with raw OGB features. Their language encoders are **not jointly fine-tuned**. SAGN/SLE learns from the 1,636-value input; Correct & Smooth refines its predictions afterward. This is feature fusion, not an average of two final predictions. [8]

The official **ogbn-products** task has 2,449,029 product nodes, 47 categories and 61,859,140 undirected co-purchase links. The implementation stores both directions, giving 123,718,280 directional edge records. The sales-rank split has 196,615 training, 39,323 validation and 2,213,091 test nodes. External text/model features are used. [1, 8]

Component / setting	Actual choice
Feature input	LD 768 + walk-text 768 + raw OGB 100 = 1,636
Text encoder	Frozen EmbeddingGemma-300m; masked mean + L2 normalization
SAGN feature hops / label hops	2 feature hops / 9 label-propagation hops
Predictor	512 hidden channels, 1 attention head, batch size 50,000
SLE stages	300, 100, 100, 100 epochs; thresholds 0.95, 0.95, 0.999
Correct & Smooth	50 correction + 50 smoothing steps; alpha 0.35 / 0.60; scale 20
Propagation normalization	DAD / DAD: symmetric degree normalization, not the toy row mean
Independent runs	10 seeds: 10 through 19; not a 10-model prediction ensemble

Measured performance of the complete pipeline

Evaluation	Accuracy: mean +/- sample SD
Validation after selected C&S	94.0584% +/- 0.0433%
Final test after C&S	87.4984% +/- 0.0490%
Published top comparator: GLEM+GIANT+SAGN+SCR	87.37% +/- 0.06% test accuracy

The point-estimate margin is **0.1284 percentage points**. This is a numerical comparison with the published leaderboard checked on 1 November 2026, not an official #1 rank or a statistical-significance claim. [8, 9]

How to interpret the evidence

No v2 ablation isolates the contribution of walk-text. Earlier v1 exploration viewed test results while choosing the pipeline. V2 used new seeds, validation-only selection and a committed lock before its final test, but inherited the already-informed method. Training labels seed graph-label features and C&S; validation labels select models. Rechecking released predictions is not fresh end-to-end retraining. [8]

8. Quick reference and sources

The same word can mean different things at different stages.

Term	Meaning in this report
Node ID / attribute	E identifies a node; Sony WF-1000XM5 is product text attached to it.
One-hot / one-hop	[1,0] encodes a class; one hop is one graph edge.
Embedding	A latent feature vector, not automatically class probabilities.
Feature concatenation	Join LD, walk-text and raw coordinates end to end: 768 + 768 + 100. No averaging or joint encoder training is implied.
Label / prediction	The known answer versus the model's estimated class scores.
LD target / SAGN weight	A training-supervision vector versus attention across feature hops.

GCN, GAT and SAGN in one comparison

Model	Where the aggregation weights act
GCN	Degree-normalized neighbor messages during graph layers.
GAT	Learned attention between individual neighbors.
SAGN	Learned attention across precomputed, encoded hop summaries.

Primary references and project evidence

[1] OGB: ogbn-products dataset and task.

[2] Shi et al. Label Deconvolution paper and author implementation / released features.

[3] Tan et al. WalkLM, NeurIPS 2023.

[4] Sun, Gu and Hu. SAGN with Self-Label-Enhanced training.

[5] Huang et al. Correct & Smooth paper, ICLR 2021.

[6] PyG source: CorrectAndSmooth (fixed-scale and smoothing behavior).

[7] PyG source: LabelPropagation (fixed restart seed).

[8] Project: README, protocol disclosure, final result JSON.

[9] Official OGB products leaderboard, checked 1 November 2026.

The central combination: released LD features supply a task-oriented view; frozen WalkLM-style features supply a walk-context view. We concatenate both with raw features, then train SAGN/SLE and apply Correct & Smooth. The score belongs to the full pipeline, not an isolated measured walk-text gain.